System Recovery & Pagefile Resurrection

This is the entire process I went through to get my system's virtual memory working again. Honestly, it was a long, chaotic, and messy ride. I truly hope you never have to deal with this, because it is definitely not fun, my pal.

What started as random application crashes quickly escalated into a deep OS corruption. Below is the complete narrative of how I diagnosed, fought, and ultimately fixed a deeply broken Windows environment.

Root Cause Analysis: What Actually Happened?
After extensive log digging, the root cause was identified: corrupted security permissions on the root directory of the C:\ drive

This permission glitch stripped away the `SYSTEM` and `Administrators` accounts ability to write to the drive's root. As a direct consequence, Windows was physically blocked from creating and persistently maintaining the `pagefile.sys` file. Without virtual memory, the system bottlenecked hard: games (like *Project Zomboid*) instantly crashed with `OutOfMemoryException` errors, the Integrated GPU (iGPU) couldn't allocate shared memory, and the Windows kernel started generating critical stability faults.

Phase 1: Diagnostics, Logs, and Scouting the Damage
Before running commands blindly, I needed to gather intel from the operating system's paper trail:
Event Viewer (`eventvwr.msc`) I isolated the Windows Logs (System) and filtered for critical events tied to `BugCheck` (Blue Screens) and `Microsoft-Windows-WER-SystemError`.
Reliability History (`perfmon /rel`): Used as a timeline to track exactly when applications and drivers started collapsing.
Boot Failures: I checked the automatic repair logs via `notepad C:\Windows\System32\Logfiles\Srt\SrtTrail.txt` to see if the boot sector was failing.
Application LogsIn: investigated specific game crash logs by auditing `%UserProfile%\Zomboid\console.txt` to confirm the memory exhaustion errors.

Phase 2: Basic System Repairs & Deployment Imaging
With the symptoms mapped out, I started patching the foundational Windows files. I launched an administrative Command Prompt to deploy the System File Checker (SFC) and Deployment Image Servicing and Management (DISM) tools:

```
cmd
:: Scanning and repairing online system files
sfc /scannow

:: Repairing the local Windows image using online corruption source files
DISM /Online /Cleanup-Image /RestoreHealth
```

When the local environment became too unstable to process repairs, I forced the system into the Windows Recovery Environment (WinRE)—either by holding `Shift` while clicking Restart on the login screen, or by forcing a hard shutdown 3 times consecutively during boot.

From the WinRE offline prompt, I ran targeted offline repairs targeting the exact Windows installation directory, using an external USB installation media as a clean repair source:
cmd
:: Offline SFC scan pointing to the system drive
sfc /scannow /offbootdir=C:\ /offwindir=C:\Windows

:: Offline DISM repair pulling healthy files from an external ESD image
DISM /Image:C:\ /Cleanup-Image /RestoreHealth /Source:esd:D:\sources\install.esd

---

Phase 3: Hard Disk Auditing & File Rescue
To rule out physical SSD failure or corrupted NTFS file structures, I ran a deep surface and file system check using CHKDSK. Since the OS volume is always in use, I tried standard flags, forced volume dismounts, and scheduled offline scans:

cmd
chkdsk C: /f /r
chkdsk C: /f /r /x

(Where `/f` fixes file system errors, `/r` hunts down bad sectors to recover readable data, and `/x` forces the volume to dismount first).

To ensure this wasn't a simple lack of physical storage capacity, I verified the absolute storage limit using `fsutil volume diskfree C:`. In worst-case scenarios during this phase, I utilized the command prompt to pull critical data off the drive before it locked up, using aggressive cloning flags:
cmd
xcopy C:\Users\YourName\Documents D:\Backup\ /E /H /C /I

Phase 4: Rebuilding Virtual Memory (The Core Fix)
Once the disk and core OS files were verified, it was time to address the ghost in the machine: the missing Pagefile. Running `dir C:\pagefile.sys /AH` confirmed my fears—the hidden system file was completely absent. Checking the configuration via `wmic pagefile list /format:list` yielded nothing.

To fix this, I manually forced Windows to register a new paging file structure using Windows Management Instrumentation (WMIC), locking it between **8GB (Initial)** and **12GB (Maximum)** to prevent dynamic sizing stutters:

cmd
:: Manually creating the file structure
wmic pagefileset create name="C:\\pagefile.sys"

:: Setting explicit boundaries (8192MB to 12288MB)
wmic pagefileset where name="C:\\pagefile.sys" set InitialSize=8192,MaximumSize=12288
```

Because the corrupted system permissions kept wiping these changes on reboot, I went nuclear. I mapped out a custom `.reg` file to forcefully inject the hex values directly into the Windows Registry hive (`HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management`), targeting the `PagingFiles` key.

If the registry injection didn't persist, I built an automated fail-safe `.bat` script running those exact WMIC commands and dropped it into the startup directory (`shell:startup`) to rebuild the pagefile every single time the user logged in.

---

Phase 5: Driver Purging and Boot Loader Optimization
Memory corruption frequently mangles display drivers and hardware abstraction layers. To eliminate overlapping variables, I booted into **Safe Mode with Networking** (via WinRE Startup Settings Option 5) to isolate the system from third-party software interference.

Using DDU (Display Driver Uninstaller), I wiped all trace elements of the graphics stack:
1. Ran DDU as an Administrator.
2. Purged the Intel Integrated GPU (iGPU) drivers completely and restarted.
3. Booted back into Safe Mode and purged the dedicated NVIDIA/AMD drivers, triggering a clean restart.
4. Installed clean drivers in exact chronological order: **Intel Integrated graphics first**, followed by the **dedicated GPU drivers** with the "Perform a clean installation" flag checked.

Simultaneously, I cleaned up the **Boot Configuration Data (BCD)** to ensure no hidden boot flags were restricting my hardware. I checked for artificial memory or core caps:
cmd
bcdedit /deletevalue truncatememory
bcdedit /deletevalue numproc
(Both returned "element not found", confirming that no bad boot limitations were actively strangling my 8GB RAM or CPU cores).

To guarantee absolute boot loader health, I rebuilt the Master Boot Record and boot sectors inside the recovery prompt:
cmd
bootrec /fixmbr
bootrec /fixboot

---
```

Phase 6: Deep System Hardening
With the architecture stabilized, I traversed through the Windows administrative utilities to ensure the pagefile settings would remain locked and protected forever:
System Properties (`sysdm.cpl`): Confirmed the Advanced Performance settings visually reflected the managed allocation.
MSConfig (`msconfig`): Stripped all non-Microsoft services and startup apps to isolate memory testing.
Local Security Policy (`secpol.msc`): Audited *Local Policies > Security Options > "Shutdown: Clear virtual memory pagefile"* and ensured it was set to **Disabled**, preventing the OS from wasting cycles wiping the pagefile block on shutdown.
Registry Editor (`regedit`): Audited `DisablePagingExecutive` (set to `0`) to make sure the Windows kernel didn't try to illegally dump executive drivers out of physical RAM.

The Verdict
After resetting the root permissions of the `C:\` drive to grant **Full Control** back to `SYSTEM` and `Administrators`, the manually configured Pagefile finally locked into place.

The system now correctly bridges physical memory overflows into the 8GB-12GB SSD buffer. Games run flawlessly, blue screens have vanished, and the environment is entirely stable.

Summary of the tool belt used:SFC, DISM, CHKDSK, BCDEDIT, DDU, WMIC, REGEDIT, MSConfig, SECPOL, Windows Memory Diagnostic (`mdsched.exe`), NTFS permission trees, and WinRE. A true system forensics operation.